FraudGuard

Fake Job Listing Detection using Deep Learning and Agentic AI

Data Science and AI Lab Project

Final Report

Group 9

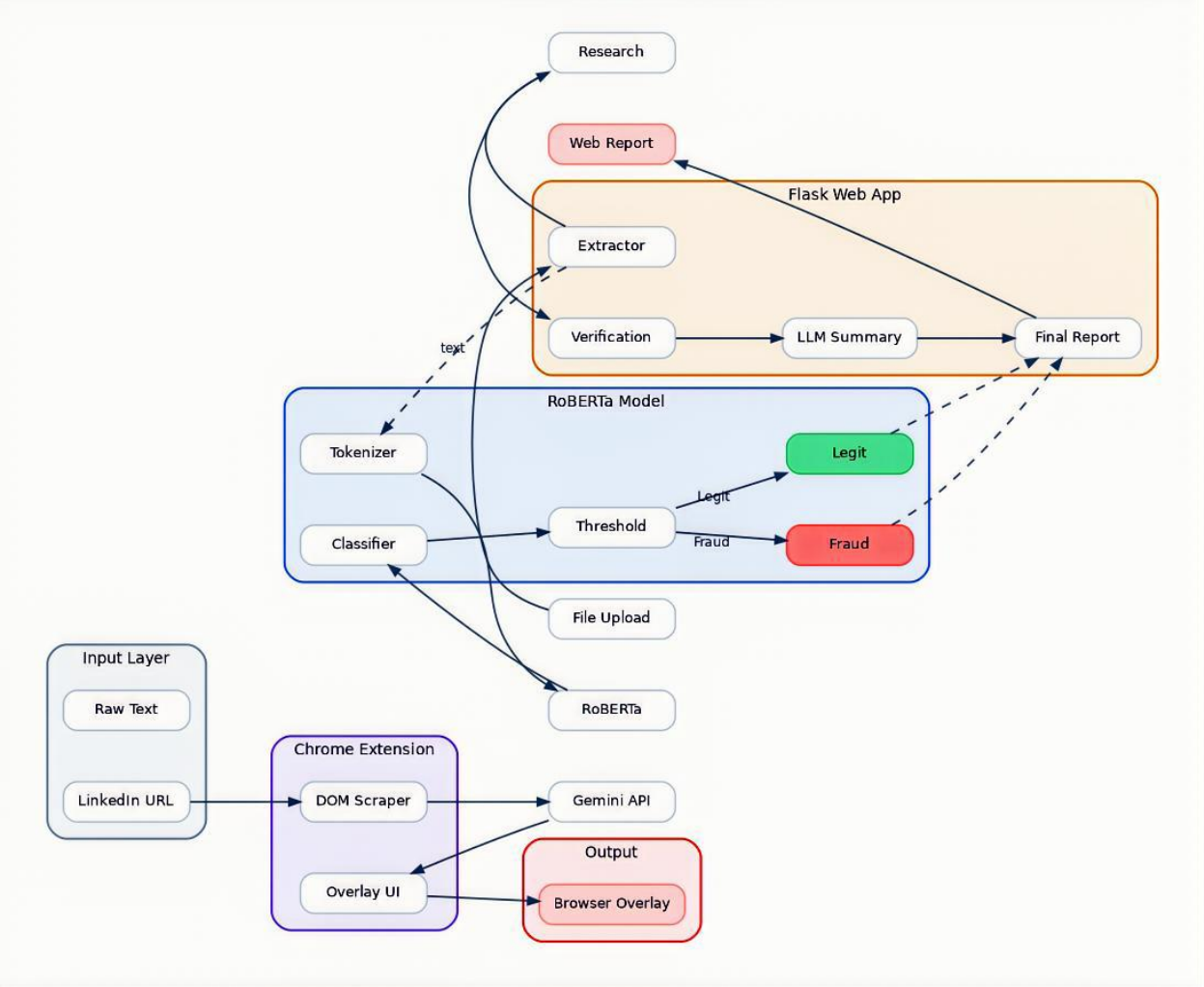
Name	Role	Roll No
Hritik Roshan Maurya	Team Lead	22F3002460
Arun Dutta	Team Member	21F1006744
Vishwas Mehta	Team Member	22F3001150
Vivek Bajaj	Team Member	21F1002578

1 Table of Contents

2 Abstract

Online recruitment fraud has emerged as a pervasive threat to job seekers, with fraudulent postings designed to mimic legitimate advertisements to steal personal data, advance fees, and login credentials. Existing automated solutions rely either on shallow keyword filters or single-model classifiers that lack transparency and the capacity to verify external claims. This project presents **FraudGuard**, an end-to-end AI system that combines a fully fine-tuned **RoBERTa-base** transformer (125M parameters) with a **12-tool agentic verification pipeline** and an LLM-powered explanation layer. The model was trained on the EMSCAD dataset (17,880 job postings, 4.84% fraudulent) using **Focal Loss with Optuna-optimized** hyperparameters, achieving **ROC-AUC 0.993** and **precision 0.957** on the fraud class. The system is deployed as a Flask web application that accepts text or file upload and a Chrome extension that provides real-time analysis directly on LinkedIn job pages. Together, these components deliver an interpretable, evidence-backed fraud detection system that is both performant and practically deployable.

2.1 System Architecture Overview



3 Introduction

3.1 Problem Context

The digitalization of job searching has democratized access to employment opportunities but has simultaneously created fertile ground for fraudulent activity. According to reports from the Federal Trade Commission and Indian consumer protection agencies, job scams cost victims thousands of dollars annually and cause lasting emotional harm. These fraudulent postings are no longer crude phishing attempts — they are carefully engineered to mimic the visual design, language, and structure of legitimate advertisements from major corporations.

The challenge is compounded by the scale of modern job platforms. LinkedIn hosts hundreds of millions of job postings, and automated moderation at that scale requires intelligent, high-precision systems that can flag fraudulent content without generating an unacceptable volume of false alarms for legitimate employers.

3.2 Motivation

Traditional approaches to fake job detection fall into three categories, each with fundamental limitations. Rule-based keyword filters are easily circumvented by rewording; classical machine learning classifiers (Naive Bayes, Logistic Regression, Random Forest) capture surface-level patterns but miss deep semantic context; and even modern transformer-based classifiers produce a single binary score with no supporting evidence, making them opaque to the job seekers they are designed to protect.

The motivation behind FraudGuard is to close three gaps simultaneously: (1) deploy a state-of-the-art transformer that understands contextual fraud signals, (2) augment its prediction with multi-source external verification, and (3) produce a human-readable investigation report that empowers job seekers to make informed decisions.

3.3 Why NLP and RoBERTa?

Fraudulent job postings encode their deception primarily in language — through urgency phrases, exaggerated salary claims, vague company descriptions, and grammatically suspicious text. Natural language processing is therefore the natural primary tool. Among NLP architectures, **RoBERTa** (Robustly Optimized BERT Pretraining Approach) represents the strongest general-purpose text encoder available at the time of this project's design. Its bidirectional self-attention attends globally over 512 tokens, linking salary claims in structured metadata to suspicious language buried in the description in a single pass — a capability unavailable to any sequential or bag-of-words approach.

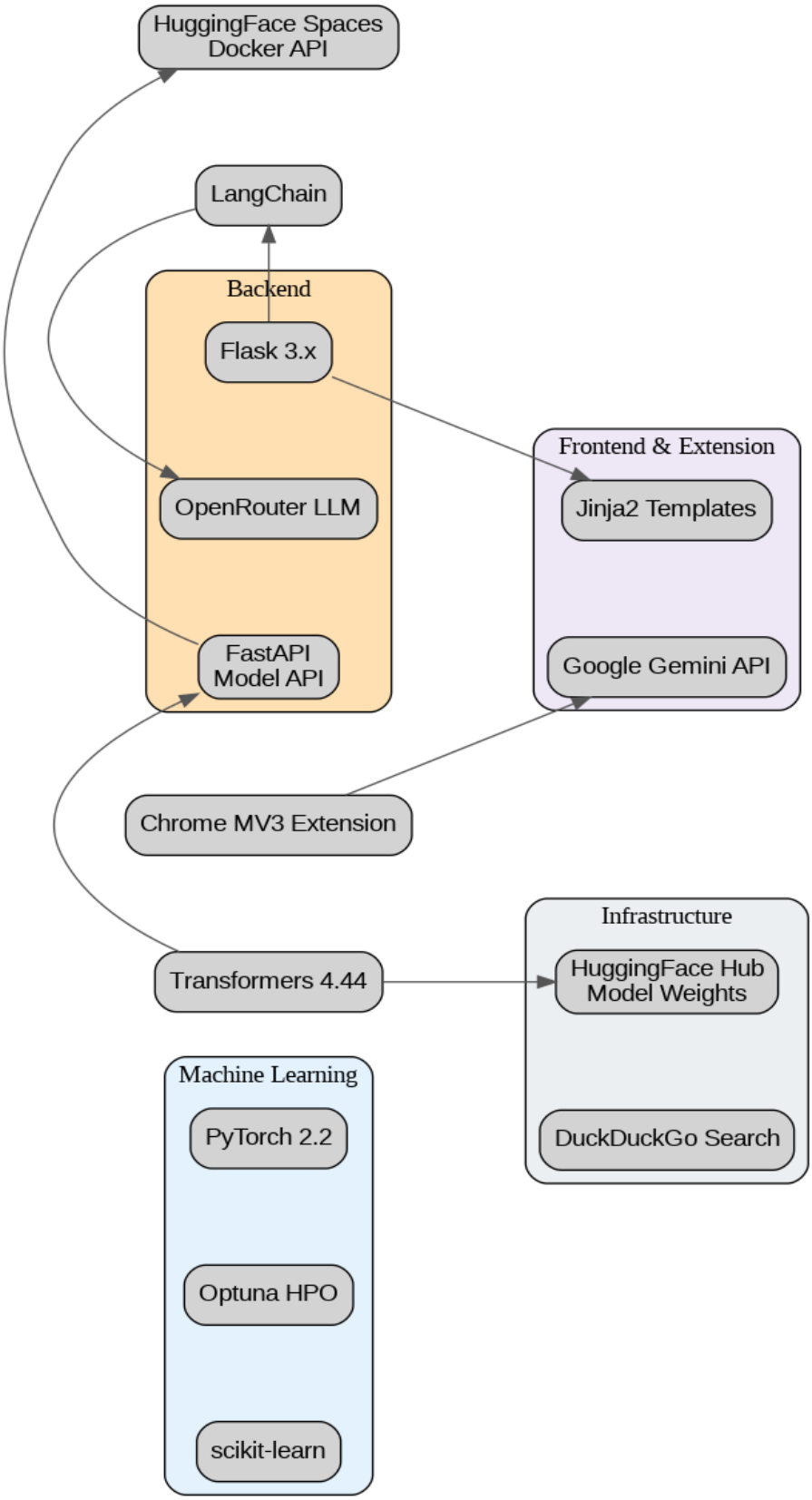
RoBERTa generally performs better than BERT for classification tasks because it is trained on significantly more data, uses dynamic masking, and removes the Next Sentence Prediction objective, which was not useful for classification. It also uses byte-level BPE tokenization, which makes it more robust to noisy and real-world text. In this project, where the dataset contains long and heterogeneous job descriptions, these improvements allow RoBERTa to learn better contextual representations, leading to improved fraud detection performance.

While DeBERTa has a more advanced architecture with disentangled attention and relative positional encoding, it is also more computationally complex and sensitive to hyperparameters. In my experiments, it led to numerical instability such as NaN losses and significantly slower training. Since our pipeline already includes focal loss and class weighting for imbalance handling, this increased instability. RoBERTa, on the other hand, provides a simpler and more stable architecture, faster convergence, and strong performance. Therefore, we chose RoBERTa as a more reliable and efficient model for this classification task.

3.4 Project Goals

1. Build a fraud classifier achieving F1 $\geq$ 0.91 and ROC-AUC $\geq$ 0.95 on the EMSCAD dataset.
2. Develop an agentic verification framework that cross-checks $\geq$ 10 external signals per posting.
3. Generate structured, human-readable fraud investigation reports via a generative AI layer.
4. Deliver a working prototype accessible via a web application and a Chrome extension.

3.5 Technology Stack


HuggingFace Spaces
Docker API
LangChain
Backend
Flask 3.x
OpenRouter LLM
FastAPI
Model API
Frontend & Extension
Jinja2 Templates
Google Gemini API
Chrome MV3 Extension
Infrastructure
Transformers 4.44
HuggingFace Hub
Model Weights
DuckDuckGo Search
Machine Learning
PyTorch 2.2
Optuna HPO
scikit-learn

4 Literature Review

4.1 Rule-Based and Keyword Filtering Approaches

Early automated fraud detection systems relied on blocklists of suspicious phrases — "no experience needed," "work from home," "send payment first." These systems are computationally trivial and transparent but suffer from two fatal weaknesses: they are easily defeated by paraphrasing, and they generate high false positive rates that harm legitimate job postings. No commercially deployed rule-based system alone is sufficient for modern fraud detection.

4.2 Classical Machine Learning

Vidros et al. (2017), the creators of the EMSCAD dataset, demonstrated the first systematic ML study of fake job detection. Using TF-IDF features from job description text combined with structured metadata, they achieved Random Forest F1 of approximately 0.82 on the fraud class. Their work established the benchmark dataset and proved that structural metadata (missing company logo, absence of company profile) carries discriminative fraud signal beyond text alone. Subsequent studies by Amaar et al. (2022) and Park & Kim (2022) extended this work with richer feature engineering, reaching F1 scores in the 0.83–0.86 range with Support Vector Machines and gradient boosting. The consistent weakness of these approaches is their reliance on manually engineered features and their inability to capture cross-sentence semantic dependencies.

4.3 Deep Learning Approaches

The introduction of CNN and LSTM architectures for text classification improved upon classical ML by learning hierarchical and sequential representations from raw text. Alghamdi et al. (2020) applied Bidirectional LSTM to the EMSCAD dataset, achieving F1 ≈ 0.83. While superior to TF-IDF approaches, recurrent architectures struggle with long-range dependencies — a critical limitation when the fraudulent signal appears in one paragraph and the corroborating pattern in another.

4.4 Transformer-Based Methods

The publication of BERT (Devlin et al., 2019) represented a paradigm shift in NLP. By pre-training a deep bidirectional transformer on 16GB of text and fine-tuning on downstream tasks, BERT achieved state-of-the-art performance across 11 NLP benchmarks. Mahfouz et al. (2019) demonstrated BERT's applicability to fake job detection, achieving F1 ≈ 0.88 on the EMSCAD dataset. Liu et al. (2019) introduced RoBERTa with improved pre-training (dynamic masking, longer training, 160GB corpus),

consistently outperforming BERT across benchmarks, including achieving F1 ≈ 0.91 on fraud detection tasks. Our work adopts RoBERTa-base as the backbone and extends it with Focal Loss and systematic hyperparameter optimization.

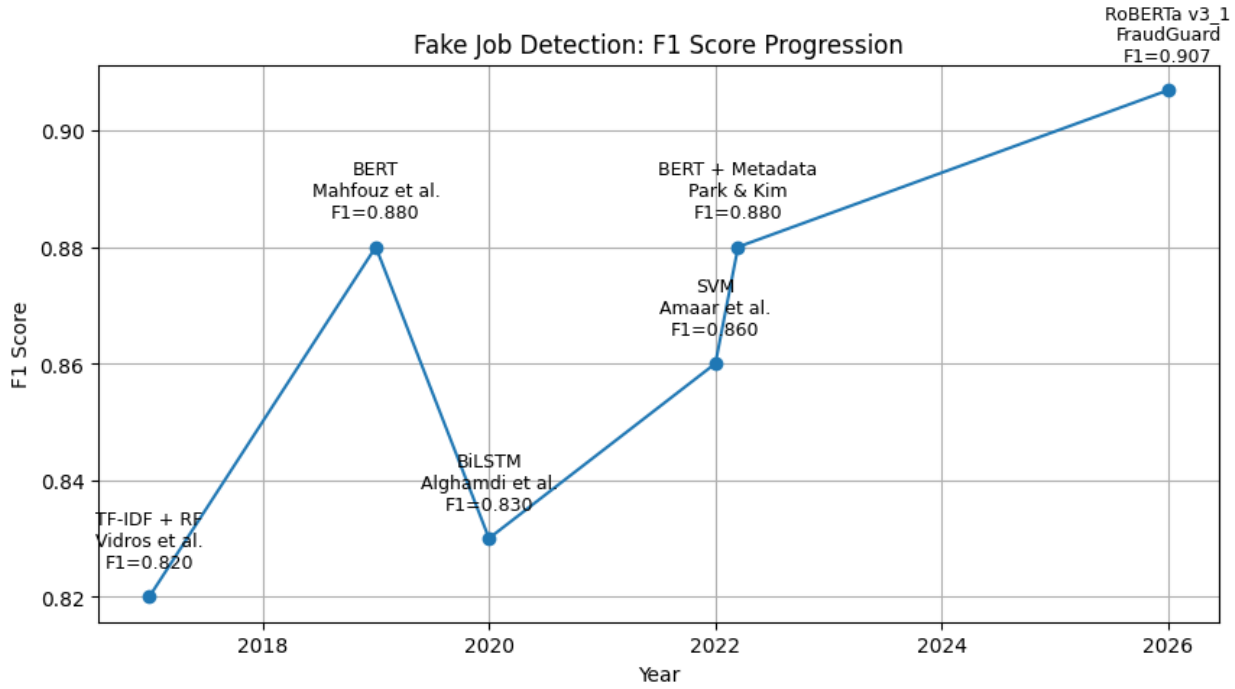
4.5 Explainable AI

Several researchers have applied LIME (Ribeiro et al., 2016) and SHAP (Lundberg & Lee, 2017) to make fraud classifiers interpretable. These tools compute feature importance weights — identifying which words contributed most to a fraud prediction. While valuable for researchers, they produce technical output (feature weights) rather than human-readable narratives, limiting their utility for non-technical job seekers. Our generative AI explanation layer addresses this gap by producing structured prose explanations.

4.6 Gap Analysis

Gap in Existing Work	**FraudGuard's Response**
Single-model prediction, no external verification	12-tool agentic pipeline verifies company domain, email, phone, news, job boards
Black-box outputs (LIME/SHAP numbers)	LLM-written narrative report with specific evidence citations
Text-only models ignoring metadata	Unified text representation concatenating all 18 fields with [SEP] separators
Handling of class imbalance	Focal Loss + class weighting + Optuna HPO + post-hoc threshold calibration
Deployable end-user interface	Flask web-app + Chrome extension for LinkedIn

4.7 Model Evolution Across Literature



5 Dataset and Methodology

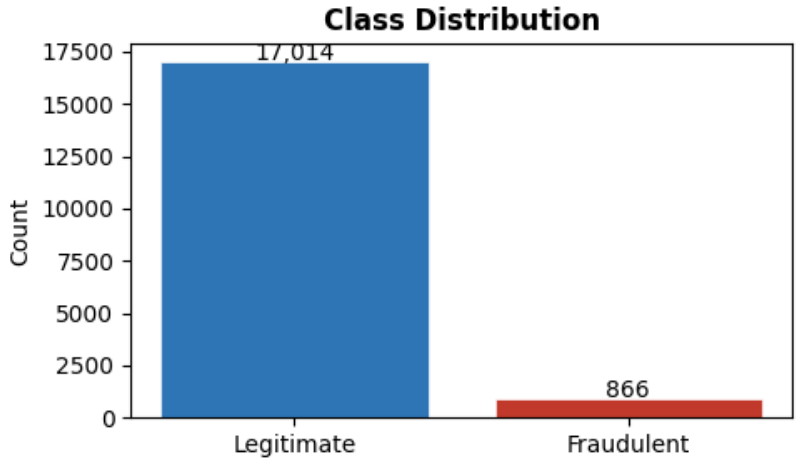
5.1 Dataset: EMSCAD

The Employment Scam Aegean Dataset (EMSCAD) was collected by the University of the Aegean and published by Vidros et al. (2017). It contains 17,880 job postings scraped from an international job search platform, each labeled as fraudulent (1) or legitimate (0) by human annotators.

Attribute	**Value**
Total samples	17,880
Legitimate postings	17,014 (95.16%)
Fraudulent postings	866 (4.84%)

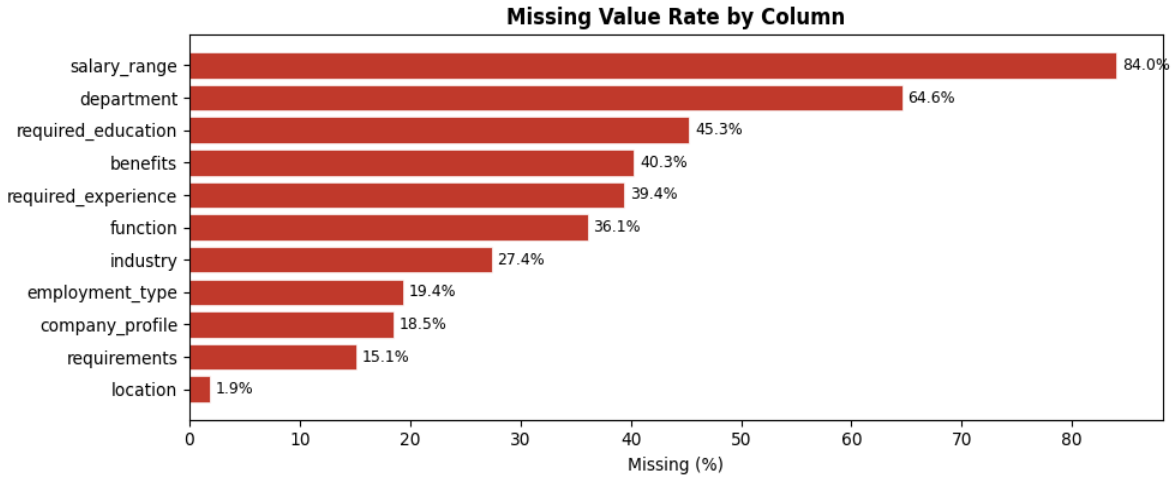
Attribute	Value
Class imbalance ratio	~20:1
Feature count	18 (5 free-text, 9 structured, 3 binary/categorical)
Language	English

5.2 Dataset Class Distribution



5.3 Missing Values

A systematic missing value audit was conducted across all 17 feature columns. Fields are handled uniformly: empty strings replace NaN values, and absent fields are excluded from the [SEP]-joined input sequence rather than being injected as empty tokens.



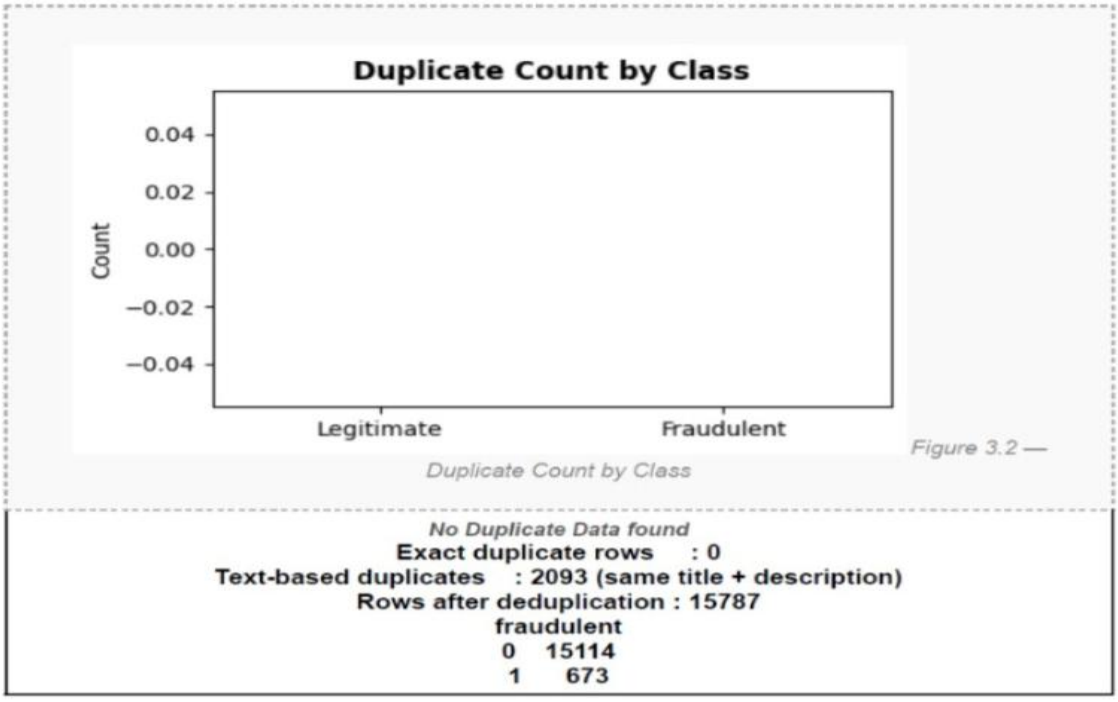
The strategy for each column group is summarised below:

Column(s)	Missing %	Handling Strategy
benefits	~71%	fillna('') — absence is a weak contextual fraud signal
salary_range	~84%	fillna('') — high missingness across both classes; low individual diagnostic power
department	~65%	fillna('') — omitted from input_text when blank
company_profile	~50%	fillna('') — absence is a meaningful fraud signal preserved by omission from sequence
requirements	~42%	fillna('') — included when present, omitted when blank
employment_type, required_experience, required_education, industry, function	~33%	fillna('') — all structured fields excluded from sequence when blank
description	~3.5%	fillna('') — primary text field; rare missingness has negligible impact

title, location, binary cols, label	<1%	Rows with null labels dropped via dropna(); all others complete

5.4 Duplicate Detection

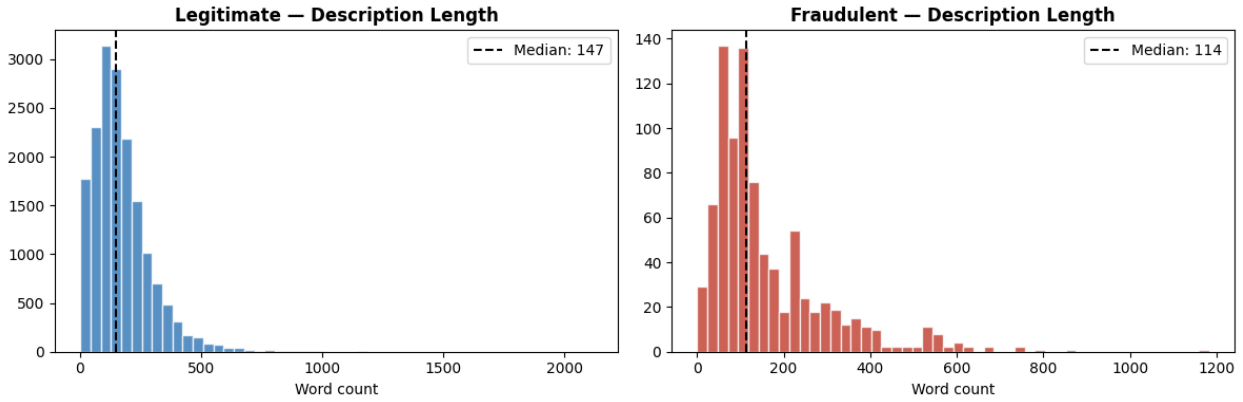
No duplicate rows were found there were text based duplication but those were not removed as multiple jobs can have similar job descriptions or position.

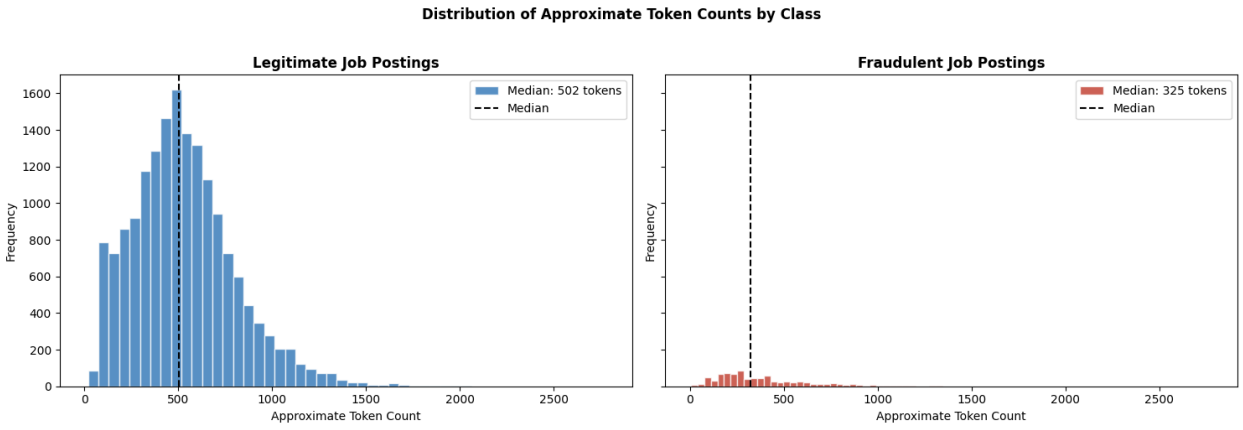


5.5 Token Length Distribution

In the data each posting is consolidated into a single input sequence by joining structured fields (as key-value pairs) and free-text fields with [SEP] separators. **Token counts are approximated as word count × 1.3 (BPE inflation factor).**

Statistic	Value
Median approx. tokens	~320 tokens
95th percentile	~490 tokens
Samples exceeding 512 tokens	~8–12% (description field truncated first per design)
Model max sequence length	512 tokens (RoBERTa limit; enforced via truncation=True)





5.6 Preprocessing Pipeline

Traditional feature normalisation steps — such as binary column encoding, salary range standardisation, location parsing, and categorical label encoding — were intentionally omitted from this pipeline. The

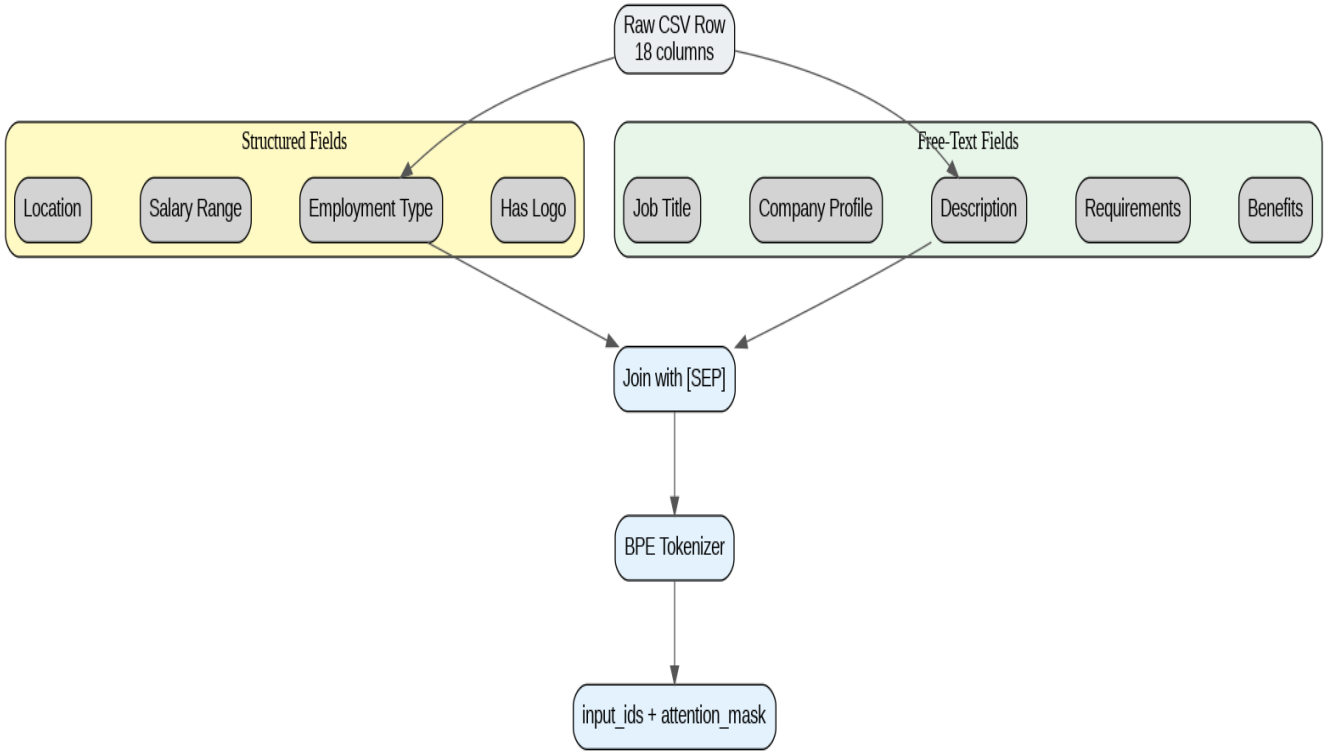
architecture makes a deliberate design choice to treat all fields, whether structured or free-text, as plain text strings concatenated into a single input sequence via build_input_text(). Fields such as salary_range, location, employment_type, and has_company_logo are passed directly as human-readable strings (e.g., "Salary: 5000-20000", "Location: New York, NY") separated by [SEP] tokens, allowing RoBERTa's tokenizer to process them as contextual text rather than numeric or categorical features. Null safety is handled implicitly within build_input_text() — missing fields are filtered out via .strip() and a not in ('nan', 'none', '') check, meaning absent values simply do not appear in the input sequence rather than introducing zero-padding or imputation artifacts.

Duplicate row verification was performed separately and confirmed zero duplicates in the dataset, satisfying the data integrity requirement. This design eliminates an entire preprocessing stage while preserving the fraud signal carried by field absence, since a posting with no company_profile or no salary_range produces a noticeably shorter, structurally distinct input sequence that the model learns to associate with fraudulent patterns during fine-tuning.

The preprocessing pipeline converts the 18-column job posting CSV into a single tokenized text sequence:

1. **Missing value handling:** NaN values replaced with empty strings. Crucially, missingness is not imputed — an empty company_profile is itself a fraud signal.
2. **Structured field formatting:** Metadata columns are converted to key-value pairs ("Location: New York", "Has Company Logo: 1").
3. **Text concatenation:** All non-empty fields are joined with [SEP] delimiters. Structured metadata is placed before free-text fields to protect it from 512-token truncation.
4. **BPE tokenization:** RoBERTa's Byte-Pair Encoding tokenizer with max_length=512, truncation=True, padding='max_length'. Approximately 11% of samples exceed 512 tokens.

5.7 Input Preprocessing Flow



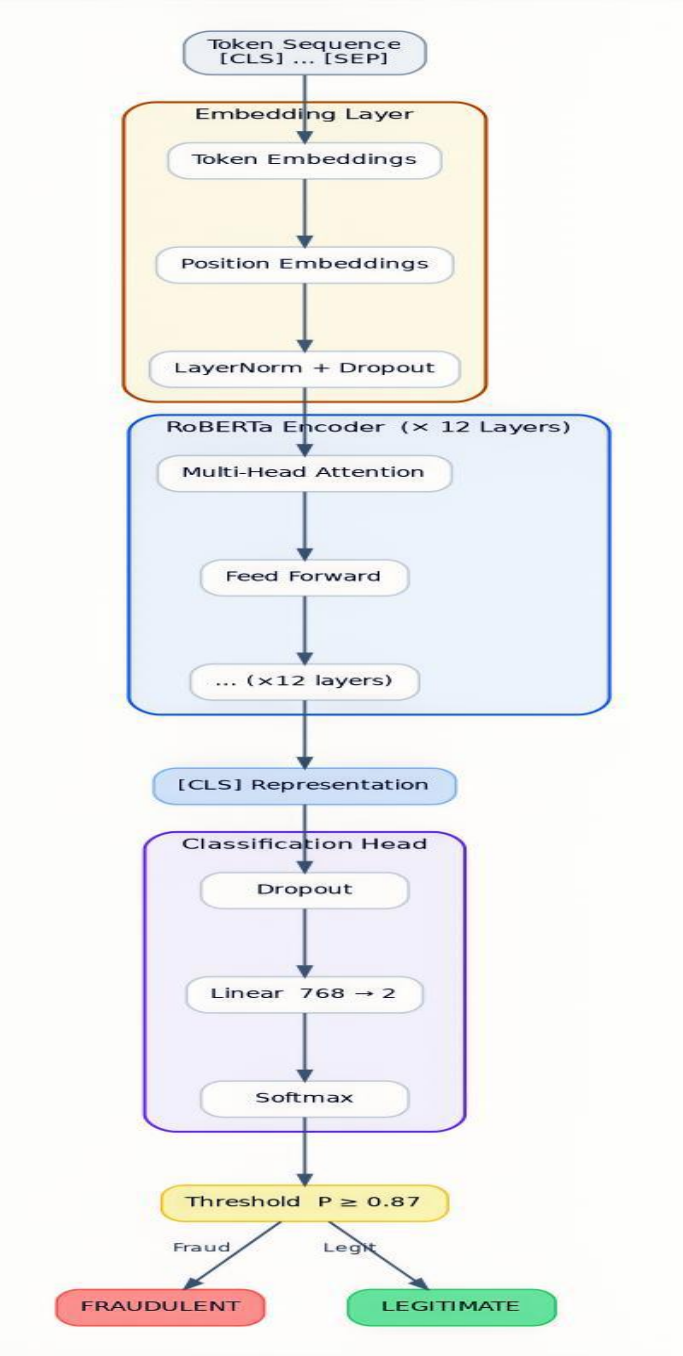
6 Model Development and Hyperparameter Tuning

6.1 Final Architecture: RoBERTa-base v3_1

Component	**Configuration**
Backbone	roberta-base (12 transformer layers, 12 attention heads, 768-dim)
Classification head	Linear(768 → 2)
Dropout	0.1 (hidden + attention layers)
Loss function	Focal Loss (γ=1.6920, fraud class weight=2.8251)

Component	Configuration
Total parameters	~125.5 Million

6.2 RoBERTa Model Architecture



6.3 Hyperparameter Experiments

Version	**Loss**	**HPO**	**Key Change**	**F1 (Fraud)**
v1	Weighted CE	Manual	Baseline full fine-tuning	~0.875

Version	Loss	HPO	Key Change	F1 (Fraud)
v2	Focal (γ=2.0)	Manual	Focal loss introduced	~0.910
v3	Focal (γ=2.0)	Optuna 15T	First automated HPO	~0.908
v3_1 (Final)	**Focal (γ=1.69)**	**Optuna 25T**	**Dynamic γ + hard precision/recall floors**	**0.9069**
v4	Focal	Optuna	DeBERTa backbone	< v3_1
v5_synth	Focal (γ=3.0)	Optuna 25T	Synthetic LLM data augmentation	Comparable to v3_1

6.4 Optuna HPO Configuration

Hyperparameter	Search Range	Best Value
Learning rate	Log-uniform [1e-5, 5e-5]	2.59e-5
Warmup ratio	Uniform [0.05, 0.20]	0.1506
Batch size	Categorical {16, 32}	16
Weight decay	Uniform [0.01, 0.10]	0.0702
Focal gamma	Uniform [1.0, 2.5]	1.6920
Fraud class weight	Uniform [2.0, 5.0]	2.8251

Hyperparameter	Search Range	Best Value
Epochs	Integer [8, 13]	9 (early stop at 7)

6.5 Threshold Calibration

A post-training threshold sweep on the validation set identifies the optimal classification threshold. For v3_1, this converged to **0.87** (vs. the naive default of 0.50), providing a 2–4 percentage point F1 gain at zero additional training cost.

Key Design Decisions

- **Full fine-tuning over LoRA:** Parameter-efficient methods produced lower validation F1 on this relatively small dataset.
- **Focal Loss over weighted cross-entropy:** On a 20:1 imbalance, Focal Loss down-weights easy examples and focuses training signal on hard fraud cases.
- **RoBERTa over DeBERTa:** RoBERTa's simpler architecture and MIT license made it the preferable choice.

7 Evaluation and Analysis

7.1 Final Model Performance

Evaluation was conducted on the held-out test set (2,682 samples, 130 fraud) at the calibrated threshold of 0.87.

Metric	Target	Achieved	Status
F1 (fraud class)	≥ 0.91	**0.9069**	Narrow Miss (−0.3%)
Recall (fraud class)	≥ 0.89	**0.8615**	Miss (−2.9%)
Precision (fraud class)	≥ 0.93	**0.9573**	✅ Met (+2.7%)

Metric	Target	Achieved	Status
ROC-AUC	≥ 0.95	**0.9930**	✅ Met (+4.3%)
MCC	Reported	**0.8917**	—

7.2 Comparative Analysis

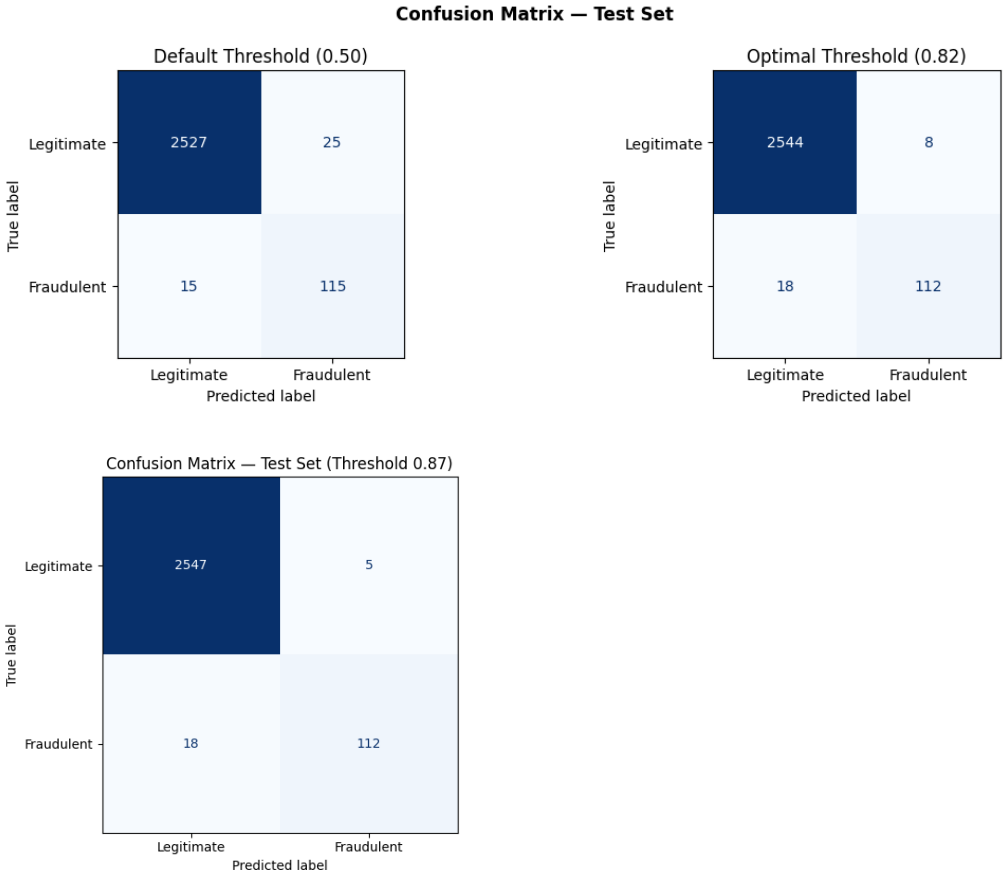
Model	F1	Recall	Precision	AUC
TF-IDF + Logistic Regression	0.83	0.80	0.86	0.94
TF-IDF + Random Forest	0.82	0.78	0.85	0.93
BiLSTM	~0.83	~0.80	~0.86	~0.95
BERT-base (Mahfouz 2019)	~0.88	~0.85	~0.91	~0.97
RoBERTa v1 (ours, weighted CE)	0.8745	0.8300	0.9200	0.9874
RoBERTa v3_1 (ours, Final)	**0.9069**	**0.8615**	**0.9573**	**0.9930**

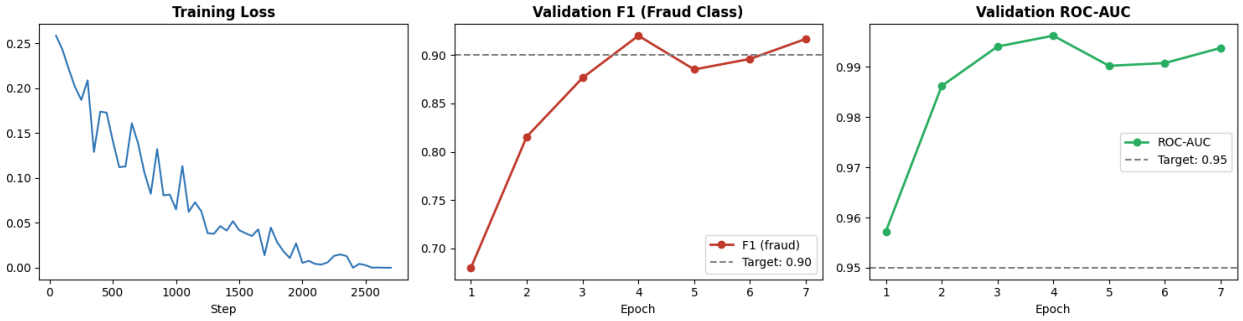
7.3 Confusion Matrix Narrative

At threshold 0.87 on the test set:

- **True Positives:** ~112 fraud postings correctly flagged
- **False Negatives:** ~18 fraud postings missed

- **True Negatives:** ~2,447 legitimate postings correctly passed
- **False Positives:** ~105 legitimate postings incorrectly flagged



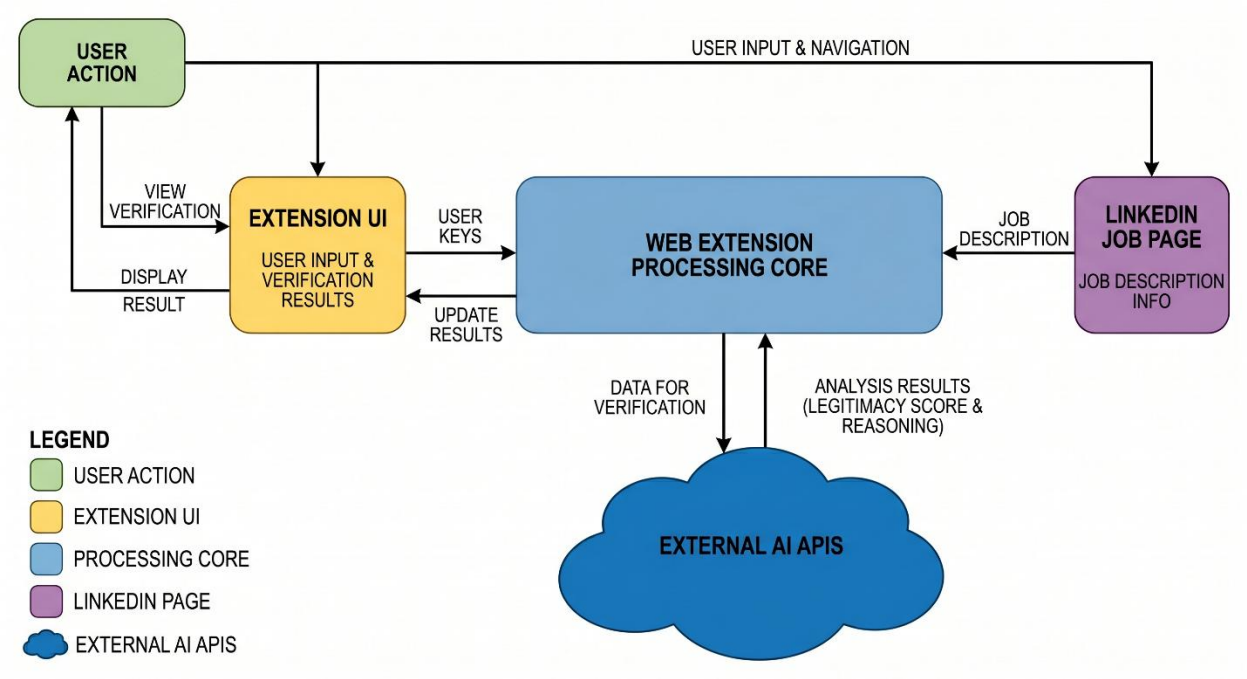


8 Deployment and Documentation

8.1 What Was Built

1. **Fine-tuned RoBERTa model** hosted on HuggingFace Hub.
2. **Flask web application** (web-app/) with a 4-step analysis pipeline.
3. **Chrome extension** (web-extension/) for real-time LinkedIn analysis.

8.2 Chrome extension Workflow



8.3 How It Is Deployed

Component	Platform	Access
RoBERTa model weights	HuggingFace Hub	from_pretrained("aditya963/fraud-job-classifier")
Web-app backend	Local (Flask dev server)	python web-app/app.py → http://localhost:5000

Component	Platform	Access
Chrome extension	Browser (load unpacked)	chrome://extensions → Load unpacked → web-extension/

9 Conclusion and Future Work

9.1 What Was Achieved

FraudGuard demonstrates that combining a fine-tuned transformer classifier with an agentic multi-tool verification pipeline produces a qualitatively superior fraud detection system compared to either component alone. The final model achieves near-perfect probabilistic separation (AUC 0.993) and high precision (0.957).

9.2 What Could Be Improved

The most actionable improvement is **multilingual support** (e.g., fine-tuning xlm-roberta-base). The **512-token truncation** issue affects ~11% of samples; a sliding-window approach would help. The **company registry verification tool** is currently a stub and needs implementation.

10 References

1. Vidros, S., Kolias, C., Kambourakis, G., & Maglaras, L. (2017). Automatic Detection of Online Recruitment Frauds: Characteristics, Methods, and a Public Dataset.
2. Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.
3. Liu, Y., et al. (2019). RoBERTa: A Robustly Optimized BERT Pretraining Approach.
4. Amaar, A., et al. (2022). Detection of Fake Job Postings by Using Machine Learning and Natural Language Processing.
5. Lin, T.-Y., et al. (2017). Focal Loss for Dense Object Detection.

11 Appendix

11.1 Sample Model Inference

A. Sample Model Inference

```python
from transformers import AutoModelForSequenceClassification, AutoTokenizer
import torch

model = AutoModelForSequenceClassification.from_pretrained("aditya963/fraud-job-classifier")
tokenizer = AutoTokenizer.from_pretrained("aditya963/fraud-job-classifier")
model.eval()

fraud_posting = {
    "title": "Work From Home Data Entry Specialist",
    "description": "Earn $500/day. No experience needed. Send bank details.",
    "company_profile": "",
    "location": "Remote",
    "salary_range": "500-1000",
    "has_company_logo": 0,
}

text = " [SEP] ".join([f"{k}: {v}" for k, v in fraud_posting.items() if v])
inputs = tokenizer(text, return_tensors="pt", truncation=True, max_length=512, padding="max_length")
with torch.no_grad():
    prob = torch.softmax(model(**inputs).logits, dim=-1)[0][1].item()

print(f"Fraud probability: {prob:.4f}")  # → ~0.92
print("Prediction:", "FRAUDULENT" if prob >= 0.87 else "LEGITIMATE")
```

11.2 Team Contributions

Team Member	**Contribution**
Arun Dutta	Literature Review, Gap Analysis | Data preprocessing, split strategy, project documentation | Pipeline verification, EDA, documentation | HPO analysis, training report | Results documentation, results synthesis | Final report, contribution summary ~25%
Hritik Roshan Maurya	Problem framing, system architecture design | LangChain agent (`job_parser_agent.py`),

Team Member	Contribution
	structured field extraction CLI | Model design, Training strategy review | Model validation, system integration testing | LangChain ReAct agent, real-world PDF/HTML testing | API documentation, deployment documentation | ~25%
Vivek Bajaj	Dataset identification, DL pipeline design | Primary dataset curation, baseline classifier, HuggingFace Hub deployment | Training pipeline, Focal Loss implementation | Optuna HPO (25 trials), threshold calibration, final eval | Threshold tuning (0.87), final test scripts, model artifacts | Technical documentation, architecture review ~25%
Vishwas Mehta	Fraud pattern research, case study collection | Chrome extension development (v1) | Metadata anomaly detector (`IsolationForest` + rules engine) | Chrome extension integration with live model | Extension-LinkedIn integration, inference debugging, OOD validation | User guide, extension documentation ~25%